

CI/CD 部署方案

高校实验室资产与报修管理平台

项目名称	高校实验室资产与报修管理平台
团队编号	第 6 组
团队成员	王旻辉、巴力江·托力肯别克、廖煌、迪丽热巴·阿布都西克尔
班级 / 学号	软件工程 2023-2 / 2311231056
二级学院 / 专业	/
授课教师	谭清萱
版本号	V1.0
日期	
编写人 / 审核人	/

本文件是源稿（Markdown）。定稿流程：AI 出初稿 → 人工改写表述（防查重）→ 补封面信息 → 导出 PDF（须嵌入字体）。

导出前请把正文中的 `_____` 与「待人工改写」标记全部清掉。

修订历史

版本	日期	修订内容	修订人
V0.1	_____	初稿完成（AI 辅助生成，内容取自仓库真实 CI 与容器化文件）	_____
V1.0	_____	人工改写表述、补齐封面信息、定稿	_____

编写说明与 AI 使用声明

本文档允许使用 AI 工具，但内容的准确性由本组负责。各章的生成方式如实声明如下：

章节	生成方式	说明
1 引言~2 环境规划	AI 生成初稿 → 待人工改写	三环境差异取自 <code>docker-compose.yml</code> 与 README
3 流水线设计	AI 生成初稿 → 待人工改写	与仓库 <code>.github/workflows/ci.yml</code> 四个 job 一一对应
4 容器化与配置管理	AI 生成初稿 → 待人工改写	取自两个 Dockerfile、 <code>nginx.conf</code> 、 <code>mysql-init.sh</code>
5 部署策略	AI 生成初稿 → 待人工改写	金丝雀选型理由为本组实际决策
6 验证与如实说明	AI 生成初稿 → 待人工改写	「已验证」与「未验证」严格分开，不粉饰

防查重提示（提交前请删除本条）：本稿依据 `ci.yml`、Dockerfile、`docker-compose.yml`、README 与 T3/T4 留痕组织，

与仓库文件存在同源表述风险。定稿前必须人工通读重写并查重。

1 引言

本文档描述「高校实验室资产与报修管理平台」的持续集成与部署方案。系统后端为 Spring Boot 2.5.15（Java 8, Maven），前端为 Vue 3.5（Vite 6, Yarn），整体通过 Docker Compose 编排为单机部署单元。

方案的核心原则有两条：构建与测试在推送时自动触发（质量门），发布动作与构建解耦（由 Nginx 上游权重控制放量）。部署策略选定金丝雀发布，理由见第 5 章。

2 环境规划（开发 / 类生产 / 生产）

维度	开发环境 dev	类生产 staging	生产 prod
用途	本地开发与联调	集成验证、答辩演示	正式对外服务
部署方式	IDEA 直跑 + <code>vite dev</code>	docker compose 单机	docker compose + Nginx 反代
数据库	本机 MySQL 8 <code>education_system</code>	容器 MySQL 8 <code>education_system_staging</code>	容器 MySQL 8 <code>education_system</code> ， 每日备份
缓存	本机 Redis	容器 Redis 7	容器 Redis 7 + 持久化卷
配置来源	<code>application.yml</code> 默认值	环境变量 <code>RUOYI_DB_*</code>	环境变量 + 密钥管理
前端 API 前缀	<code>/dev-api</code>	<code>/stage-api</code>	<code>/prod-api</code> （Nginx 反代并剥离前缀）
日志级别	<code>com.ruoyi: debug</code>	<code>com.ruoyi: info</code>	<code>com.ruoyi: warn</code>
访问控制	无	内网 + 基础认证	公网 + HTTPS

三环境差异的技术落地：`application*.yml` 配置文件在三个环境保持完全一致、一字不改，差异全部通过环境变量注入（`RUOYI_PROFILE`、`SPRING_REDIS_HOST/PORT`、`RUOYI_DB_URL/USERNAME/PASSWORD`）。生产与类生产在基础

编排上再叠一层 `docker-compose.prod.yml`（端口只绑回环、口令必填、资源上限、`restart: always`）。

3 流水线设计

3.1 全链路

代码提交 → 后端构建与测试 → 前端生产构建 → 打包产物 → 镜像构建 →（部署时）Nginx 权重放量

流水线定义在 `.github/workflows/ci.yml`，由 GitHub Actions 执行，触发条件为推送到 `main`、发起 Pull Request、以及手动触发（`workflow_dispatch`，答辩演示时可免提交直接跑一次）。

3.2 四个 job

job	阶段	命令	产物	依赖
backend	后端构建与测试	<code>mvn -B clean test</code> （JDK 8 temurin）	surefire 测试报告	—
frontend	前端生产构建	<code>yarn install --frozen-lockfile</code> + <code>yarn build:prod</code> （Node 22）	<code>dist/</code>	—
package	打包可运行 jar	<code>mvn -B package -DskipTests</code>	<code>target/ruoyi.jar</code>	backend, frontend
docker	镜像构建	<code>build-push-action</code> （buildx，只 build 不 push）+ <code>docker compose config</code> 语法门禁	后端/前端两个镜像	package

几个设计要点：

- 测试绝不可跳过：`backend` job 的测试步骤不加 `-DskipTests`；`package` job 里的 `-DskipTests` 是合法的——测试已在前置 job 跑过并通过，打包阶段只需产出 jar，避免重复执行。上传 surefire 报告用 `if: always()`，测试失败时报告更要留下以便定位。
- 凭据零明文：`ci.yml` 内不出现任何密码，将来推镜像走 GitHub Secrets。
- 前端必须走 Yarn：仓库只有 `yarn.lock` 没有 `package-lock.json`（若依源码的历史状态），`npm ci` 会因缺 lock 文件失败，故用 `corepack enable` + `yarn install --frozen-lockfile`。
- 镜像只构建不推送：推送到 registry 需要凭据，答辩演示用本地镜像即可。

3.3 本地等价流水线

真实流水线跑在 GitHub Actions；本地/答辩自检用 `deploy/local-pipeline.ps1` 按相同阶段顺序跑一遍，共 6 个阶段与上面 job 一一对应。实测结果：

PASS 4 / SKIPPED 2 / FAIL 0	总耗时 216.2 s
-----------------------------	-------------

阶段 1~4（版本信息 / 后端构建与测试 / 前端构建 / 打包产物）全 PASS，产物 `ruoyi.jar` 85.1 MB；阶段 5~6（镜像构建 / 冒烟测试）因本机未安装 Docker 打印 `SKIPPED` 并继续——不伪装成 PASS。任一阶段失败即 `exit 1`。

4 容器化与配置管理

4.1 服务编排

服务	镜像 / 构建源	关键设计
mysql	<code>mysql:8.0</code> （锁版本）	首次启动按 <code>deploy/mysql-init.sh</code> 显式顺序导入 7 个 SQL；时区用数字偏移 <code>+08:00</code>
redis	<code>redis:7-alpine</code>	若依强制依赖；命名卷持久化
lab-backend	<code>./RuoYi-Vue-fast</code>	非 root（uid 1000）、 <code>TZ=Asia/Shanghai</code> 、 <code>/dev/tcp</code> 健康检查、exec 形式 ENTRYPOINT

服务	镜像 / 构建源	关键设计
lab-frontend	./RuoYi-Vue3	nginx 托管 <code>dist/</code> ， <code>/prod-api/</code> 反代后端并剥离前缀

4.2 SQL 初始化顺序（本项目最容易踩的坑）

不要将 `sql/*.sql` 直接挂进 `/docker-entrypoint-initdb.d/`——MySQL 官方镜像按文件名字母序执行初始化脚本，`laboratory_*` 会全部排在 `ry_*.sql` 之前，导致每个脚本都报「表不存在」。本仓库的解法是只挂一个显式排序的 `mysql-init.sh`，按 `[N/7]` 顺序执行 7 个脚本，且每个脚本自带序号标记、与脚本内 `for` 循环三方互校。

另一个被排除的文件是 `quartz.sql`：后端 `ScheduleConfig.java` 整个类被注释、`resources/` 下无 `quartz.properties`，Quartz 走 Spring Boot 默认内存存储（`RAMJobStore`），`QRTZ_*` 表根本不需要，故不纳入初始化顺序。

4.3 nginx 反代的两个关键点

- `proxy_pass http://lab-backend:8080/` 的末尾斜杠不能删——它是「剥离 `/prod-api/` 前缀」的关键，漏了会导致所有接口 404。
- `client_max_body_size 20m` 必须显式给出——nginx 默认仅 `1m`，而系统支持上传故障照片（后端允许单文件 10MB / 请求 20MB），缺了这行会拦截成 413，且后端日志毫无痕迹。

4.4 配置与密钥管理

数据库/Redis/上传路径等差异全部经环境变量注入（见第 2 章），密钥经 `.env`（已被 `.gitignore` 忽略，不入库）与 GitHub Secrets 管理。生产镜像构建为两段式（Maven 构建 → JRE 8 运行），运行用户为非 root 的 `uid 1000`。

5 部署策略：金丝雀发布

在滚动 / 蓝绿 / 金丝雀三种常见策略中，本项目选定金丝雀发布，理由是排除法：

- 滚动发布依赖容器编排平台的滚动更新与就绪探针，本项目是 docker compose 单机部署、无 K8s，编排能力不具备。
- 蓝绿发布在单机上需同时维持两套完整实例，资源翻倍；且若依用 Redis 存登录态、两套实例会话不共享，切换时会把正在报修的用户踢下线。
- 金丝雀发布只需在 Nginx `upstream` 里给新版本一个较小权重（如 `5% → 50% → 100%`），把少量流量导入新版本观察；出错时把权重改回 0 即可秒级回滚，无需额外基础设施。

放量窗口建议 10~15 分钟观察错误率与关键接口耗时，回滚阈值为「错误率超过 1% 或出现 5xx」即回退权重到 0。放量与回滚动作由 Nginx 配置控制，不属于 GitHub Actions 流水线的职责——构建负责「产出一个可信的镜像」，发布负责「控制流量」，二者解耦。

6 验证与如实说明

6.1 已验证（两层证据，共 117 条断言全绿）

- 配置静态自治性：`.workbuddy/tools/t4_verify.py`，94 条断言，覆盖 YAML 可解析、SQL 顺序与磁盘文件及脚本序号三方互校、nginx 前缀剥离与上传上限、Dockerfile 非 root 与健康检查、口令无明文、`.dockerignore` 该排的排了。
- 环境变量绑定实证：`.workbuddy/tools/t4_env_binding_check.py` + `T4EnvBindingProbe.java`，23 条断言，用 Spring 真实属性源机制跑两套不同取值做对照——`raw`：视图证明配置文件未被改动、`eff`：视图证明 `RUOYI_PROFILE` / `SPRING_REDIS_HOST` 等确实盖得住写死的值，且 `application*.yaml` 一字未改。

6.2 未验证（环境不具备，如实说明）

本机未安装 Docker，`docker compose up` 未实际执行，因此「容器真的能起、健康检查能通过、`docker compose ps` 显示 healthy」这一层尚未经过运行验证。上文 94+23 条断言证明的是「配置自治 + 覆盖生效」，不等于「容器起来了」。

任务书中要求的 `docker compose ps` 运行截图因此无法产出——本方案以「静态断言 + 环境变量绑定实证」作为替代证据，并在此明确区分「已证明的」与「未验证的」。完整运行验证留待具备 Docker 环境的机器执行，答辩口径按此实情说明。

6.3 任务卡本身的两处修正（如实记录）

撰写本方案时发现任务卡示例有两处缺口，产出文件均已修正：

缺口	照抄的后果	修正
环境变量清单漏 <code>redis</code> 主机/端口	容器能启动但登录页出不来验证码	补 <code>SPRING_REDIS_HOST/PORT</code> 覆盖
nginx 示例漏 <code>client_max_body_size</code>	传故障照片被拦成 413	server 级与 <code>/prod-api/</code> 级各补 <code>20m</code>